

Workflow Anatomy: Inputs, Transformations, Outputs

FRIDAY AI Education — Episode 022 · 2026-07-08 · Printable Companion

Abstract

Every company is a collection of processes wearing a trench coat. Before you can automate a process, delegate it, hire for it, or hand it to an AI agent, you have to be able to see it — cleanly, without hand-waving. This paper gives you one durable lens for doing that: any process, no matter how vague it feels, can be decomposed into **inputs, transformations, outputs, owners, handoffs, and failure points**. Master this decomposition and you gain a superpower that compounds: you can walk into any operation — sales research, content production, support triage — and within an hour describe how it actually works, where it breaks, and which piece a machine could safely take. We build the vocabulary from first principles, run it against three concrete workflows, connect it to how AI-native organizations are actually wired, and end with the failure modes that separate people who *talk* about automation from people who ship it. The one non-obvious claim: the parts of a workflow that are hard to automate are almost never the transformations. They are the handoffs.

1. Why this is the first thing worth learning

There is a temptation, when you are strategically strong and technically uneven, to skip straight to the exciting vocabulary — agents, embeddings, retrieval, fine-tuning. Resist it for one more episode. All of those tools operate *on* workflows. If you cannot describe the workflow with precision, you are buying power tools before you have measured the wall.

Here is the practical stakes. When you sit across from a serious operator — a potential hire, a potential acquisition, a technical co-founder — and you say "we should automate our sales research," you have said almost nothing. The operator's next four questions will be: *What goes in? What comes out? Who owns it today? Where does it currently break?* If you can answer those crisply, you are having an engineering conversation. If you cannot, you are having a vibes conversation, and serious people can tell the difference in about ninety seconds.

The lens in this paper is the thing that lets you answer those four questions for *any* process on demand. It is not sophisticated. It is not proprietary. It is simply rigorous, and rigor is rarer than sophistication.

There is a second reason this matters specifically for building an AI-native company. Software workflow engines — the systems that run business processes at scale — were forced to make this anatomy explicit decades ago, because a machine cannot execute a process it cannot represent. The concepts you are about to learn are not a teaching metaphor. They are close to the literal data model that production orchestration systems use. When we reach the reading guide, you will see the same six ideas staring back at you from Temporal's documentation, wearing slightly different clothes.

2. The core concept, in plain language

A **workflow** is a process that turns something into something else through a sequence of steps.

That is the whole definition. Do not let anyone inflate it. The reason it is powerful is not that it is deep; it is that it is *complete*. Everything interesting about a process fits into six slots, and if you can fill all six, you understand the process well enough to change it.

Let us name the six slots, then spend the rest of the paper earning them.

1. **Input** — what the workflow consumes to get started.
2. **Transformation** — the work that changes the input into something more valuable.
3. **Output** — what the workflow produces.
4. **Owner** — the specific person, team, or system accountable for a step.
5. **Handoff** — the moment work passes from one owner to the next.
6. **Failure point** — a place where the workflow can break, stall, or produce garbage.

Three of these (input, transformation, output) describe the *flow of value*. Three of them (owner, handoff, failure point) describe the *organizational reality* the flow runs through. Beginners map the first three and declare victory. The first three are the easy half. The money — and the risk — is in the second three.

3. The mechanism underneath

3.1 Inputs: the thing you forget to specify

An input is whatever a workflow needs before it can begin. The trap is that most inputs are *implicit*. A salesperson doing research does not think "my input is a company name." They just start typing. But the company name is the input, and so is the salesperson's tacit knowledge of what a good prospect looks like, and so is their access to the tools they use. Un-named inputs are the most common reason a workflow that "works" for a human fails when you try to hand it to someone — or something — else.

A useful discipline: for every input, ask three questions. *Where does it come from? What state must it be in? What happens if it's missing or wrong?* An input you cannot answer those three questions about is an input you do not actually control, and an uncontrolled input is a failure point wearing a disguise.

3.2 Transformations: where value is added

A transformation is the step that changes the input into something more valuable than it was. Writing a summary from raw notes is a transformation. Scoring a lead from a company profile is a transformation. Classifying a support ticket by urgency is a transformation.

Two properties of a transformation matter enormously for whether you can automate it:

- **Is it deterministic or judgment-based?** "Convert this date to ISO format" is deterministic — same input, same output, every time. "Decide whether this customer is angry enough to escalate" is judgment-based. Deterministic transformations have been automatable for fifty years. Judgment-based transformations are exactly the ones large language models newly let you approach — and exactly the ones that require governance, because the model can be confidently wrong.

- **Is it reversible?** "Draft an email" is reversible — a human reads it before it sends, so a bad draft costs nothing. "Send the refund" is not reversible. The reversibility of a transformation determines how much you can trust it to run unattended. We will return to this; it is the single most important safety property in the paper.

A word of precision, since we are building foundations. A large language model performing a judgment-based transformation is not *thinking* about your customer. It is generating the statistically likely continuation of a prompt, shaped by patterns in its training. Sometimes that continuation is indistinguishable from good judgment. Sometimes it is fluent nonsense. Treating the two as the same category — "the AI decided" — is how people get burned. The model is a transformation engine with real limits, not a colleague.

3.3 Outputs: the thing that must be usable by the next step

An output is what the workflow produces. The discipline here is to define the output by *what the next step needs*, not by what is convenient to produce. A sales-research workflow whose output is "a document of interesting facts" has failed even if the facts are correct, because the next step — the salesperson writing an email — needs a *specific, structured* thing: the hook, the pain point, the person to contact. An output that is not shaped for its consumer is technically a deliverable and practically a handoff failure. Which brings us to the two slots that do the real work.

3.4 Owners: accountability is not a group activity

An owner is the single entity — person, team, or system — accountable for a step. The word "single" is load-bearing. A step owned by "the team" is a step owned by no one, and steps owned by no one are where work goes to die silently. When you map a workflow, assigning an owner to each step is not bureaucracy; it is the act of discovering which steps currently have no owner. Those uncovered steps are almost always where the process is quietly failing.

In an AI-native organization, owners get more interesting, because an owner can be a machine. But — and this is a rule, not a preference — a machine owner still reports to a human owner. "The agent owns lead scoring" is only a coherent sentence if there is also a human accountable for the agent scoring leads correctly. Automation moves the labor; it does not move the accountability.

3.5 Handoffs: the expensive part

A handoff is the moment work passes from one owner to the next. It is the most under-appreciated concept in this entire paper, so let us be blunt: **handoffs are where workflows actually break**. Not the transformations. The gaps between them.

Consider why. Inside a single owner's control, context is free — the salesperson knows what they were doing, remembers the prospect, has the tab open. At a handoff, context must be *reconstructed* by whoever picks the work up. Every handoff is a small act of translation, and every translation loses information. The support ticket arrives at Tier 2 with half the story. The research doc lands in the writer's inbox with no note about the deadline. The lead score shows up in the CRM with no explanation of why it's a 7.

Handoffs fail in three characteristic ways, worth memorizing:

- **Dropped** — the work arrives but no one picks it up. (No owner on the receiving side.)
- **Garbled** — the work arrives but the context does not. (The output wasn't shaped for the consumer.)
- **Delayed** — the work arrives eventually, but the latency breaks the value. (A support triage that's correct but six hours late is a failure.)

If you internalize one thing from this episode, make it this: when you design a workflow — human or automated — you spend most of your design effort on the handoffs, not the steps. The steps are the parts everyone already understands. The handoffs are the parts everyone assumes will "just work," which is precisely why they don't.

3.6 Failure points: name them before they name you

A failure point is any place the workflow can break, stall, or produce something wrong. Some live inside transformations (the model hallucinates a fact). Most, as we've said, live at handoffs. A mature map of a workflow does not just show the happy path; it annotates each step and each handoff with *how it fails and what happens when it does*. This is not pessimism. It is the difference between a process you can trust to run and a process that has simply not broken yet.

4. The anatomy on one page

Before the worked examples, here is the whole framework in a form you can pin to a wall.

Slot	Question it answers	Common mistake	Automation flag
Input	What does this consume, and in what state?	Leaving tacit inputs unnamed	Missing/malformed inputs are the #1 automation failure
Transformation	What work changes input into value?	Assuming judgment is deterministic	Deterministic → safe; judgment-based → needs governance
Output	What does the <i>next step</i> need?	Defining output by what's easy to make	Must be structured for machine or human consumer
Owner	Who is single-throat accountable?	"The team owns it" (i.e. no one)	A machine owner still needs a human owner above it
Handoff	Where does work + context change hands?	Assuming context transfers for free	Handoffs, not steps, are where automation breaks
Failure point	How does this break, and then what?	Mapping only the happy path	Reversible failures are safe to automate; irreversible ones need a gate

5. Three workflows, dissected

Theory earns its keep on real processes. Here are three you will meet in any company, mapped with the same lens. Notice that the interesting observations are always about handoffs and failure points.

5.1 Sales research

The naive description: "We research prospects before reaching out."

The anatomy:

- **Input:** A company name or domain, plus an implicit definition of "good prospect" (which usually lives only in the rep's head — flag that immediately).

- **Transformations:** (1) Gather public signals — website, funding, headcount, recent news. (2) Judge fit against the ideal customer profile. (3) Find the specific hook — the reason to reach out *now*.
- **Output:** Not a pile of facts. A structured brief: who to contact, the one relevant pain point, the timely hook, and a fit score with a reason.
- **Owners:** Today, usually one rep owns all three transformations. That's why it doesn't scale — it's three different kinds of work fused into one person.
- **Handoffs:** The critical one is research → outreach. If the brief is a wall of text, the rep writing the email re-does the judgment work, and the research was theoretically completed but practically wasted.
- **Failure points:** Stale data (the funding round was last year). Judgment drift (every prospect scores a 7 because the rep is optimistic). And the classic when this is automated: an AI enriching the profile with a *plausible* fact that is simply false. The public web is full of confident wrong answers, and a generation engine will happily repeat them.

The lesson this example teaches: the two gathering-and-formatting transformations are largely mechanical and highly automatable. The fit judgment is not — it encodes strategy. A sane first automation takes the mechanical two and *leaves the judgment, with its irreversible consequences, to a human*. This is the wedge, not the whole leg.

5.2 Content workflow

The naive description: "We turn ideas into published posts."

The anatomy:

- **Input:** A topic or angle, a target audience, a house style, and — the input everyone forgets — the source material the piece is supposed to be grounded in.
- **Transformations:** Outline → draft → edit → fact-check → format → publish.
- **Output:** A published piece. But note the *intermediate* outputs: the outline is an output consumed by the drafter; the draft is an output consumed by the editor. A content workflow is really a chain of mini-workflows, each with its own handoff.
- **Owners:** Often several — a writer, an editor, a fact-checker, whoever holds the publish button. Every boundary between them is a handoff.
- **Handoffs:** Outline → draft is where intent gets lost ("this isn't what I meant"). Draft → fact-check is where things get *skipped* under deadline pressure, which is exactly when errors ship.
- **Failure points:** The reversible ones (a weak outline, an awkward draft) are cheap — a human catches them at the next step. The irreversible one is **publish**: once it's public, a hallucinated statistic or a plagiarized line is a real cost with your name on it.

The lesson this example teaches: this is a workflow where a generation engine is genuinely strong at several transformations — outlining, drafting, reformatting — *and* where the last handoff must remain a human approval gate. The pattern "automate the reversible middle, keep a human on the irreversible end" is not a compromise. It is the correct design.

5.3 Customer support triage

The naive description: "We route incoming tickets to the right place."

The anatomy:

- **Input:** An inbound message — often unstructured, emotional, and ambiguous. Real inputs are messier than the diagram suggests, and triage is the workflow where that mess is most obvious.
- **Transformations:** (1) Classify by topic. (2) Assess urgency. (3) Route to the right owner. Note that (1) and (3) are close to deterministic; (2) is judgment-based and is where errors concentrate.
- **Output:** A tagged, prioritized ticket in the correct queue — shaped so the receiving agent has what they need to act.
- **Owners:** Triage itself has an owner (increasingly a system); each destination queue has its own.
- **Handoffs:** Triage → resolution queue is the load-bearing one. A misrouted ticket isn't just delayed — it lands with someone who can't help and has to hand it off *again*, compounding the delay.
- **Failure points:** Misclassification sends the ticket to the wrong queue. Under-scored urgency buries an outage behind a password reset. And the automation-specific one: an over-trusted classifier that routes confidently and wrongly, at scale, with no one checking — the failure that turns a small model error into a large operational one.

The lesson this example teaches: triage is often the *best* first automation in a company, because most of it is reversible (a misroute can be corrected) and high-volume (small time savings compound). But it demonstrates the governance requirement vividly: you want the machine's confidence measured against reality, and a path for low-confidence tickets to reach a human. Which is the whole topic of the next section.

6. Connecting the anatomy to AI-native organizations

Now the payoff. Everything above is true for a company staffed entirely by humans. What changes in an AI-native organization is not the anatomy — it is *who or what can occupy each slot*.

In a traditional org, every owner is a person and every transformation is human labor. In an AI-native org, a transformation can be performed by a reasoning-and-generation engine, an owner can be a software system, and a handoff can be an API call instead of a Slack message. The anatomy is the constant; the substrate underneath each slot is the variable. This is why learning the anatomy first is not a detour — it is the coordinate system that makes the rest legible.

Two pieces of that substrate are worth naming precisely, because they are what make automated transformations *reliable* rather than merely *possible*.

Context and memory. A generation engine, on its own, knows nothing about your company — it produces plausible text from general patterns. To perform a real transformation well, it needs the relevant facts in front of it: this customer's history, this company's style guide, this prospect's actual funding data. Supplying those facts at the moment of the transformation is what a knowledge-and-memory substrate is *for*. In FRIDAY's design, GBrain plays this role — it is the memory layer that holds context so transformations run against reality instead of against the model's best guess. When you hear "retrieval augmented generation" — RAG, and it is worth saying the full phrase once so the acronym has meaning — this is the mechanism it names: retrieve the relevant facts, then generate. It works by finding stored material that is *semantically similar* to the task at hand — close in meaning, measured as distance between numeric representations — not by any literal lookup. And a caution that belongs in your foundations: those numeric representations capture *similarity*, not *truth*. Retrieving a passage because it is close in meaning is not the same as retrieving a passage because it is correct. The retrieval narrows the field; it does not certify the answer.

Governance. The moment a transformation runs without a human doing it, six things stop being optional: the context it runs against, the memory of what it did before, the tools it's allowed to touch, the permissions bounding those tools, the *evaluations* — the tests that measure whether it's actually performing well — the audit trail of what happened,

and the approval gate a human holds over irreversible steps. An automated workflow without those is not efficient; it is an unmonitored liability that has not embarrassed you yet. This is the difference between an agent and a magic employee. There is no magic employee. There is a bounded system with permissions and a paper trail, and the boundaries are the product.

This connects directly to the first practical wedge worth building: a public-content knowledge base — taking a person's or company's public body of work and making it searchable, reusable, cited, and operationally useful. Look at that through the anatomy. **Input:** a public corpus. **Transformations:** ingest, structure, index by meaning, retrieve on demand. **Output:** cited, reusable answers. **Owner:** the system, under a human. **Handoff:** retrieval → generation → the person who acts on it. **Failure point:** retrieving something merely similar and presenting it as authoritative. It is a good first wedge precisely *because* its failure mode is bounded — a wrong citation is embarrassing but reversible — and its value compounds. It is a workflow you can map completely, which is why you can build it safely.

7. The main limitation: reversibility is the whole game

If this paper has one governing idea beneath the vocabulary, it is this: **the safety of automating a transformation is determined by the reversibility of its failure, not the difficulty of the task.**

A hard transformation with a reversible failure — drafting a complex document a human will review — is *safe* to automate, because the review step catches errors before they cost anything. An easy transformation with an irreversible failure — sending a payment, publishing a post, deleting a record — is *dangerous* to automate, even though the task itself is trivial, because there is no step after it to catch the error.

This inverts most people's intuition. They assume you automate the easy things first and work up to the hard things. The correct frame is: you automate the *reversible* things first, regardless of difficulty, and you put human approval gates on the *irreversible* things, regardless of how simple they look. The gate is not there because the machine is dumb. It is there because the consequence is permanent, and permanent consequences deserve a human name attached to them.

Every one of our three examples obeyed this. Draft the email, but let a human send it. Draft the post, but let a human publish it. Triage the ticket freely, because a misroute can be undone. The pattern is not a limitation to apologize for. It is the design.

The honest boundary of this whole approach: the anatomy tells you how to see a workflow and where it is *safe* to intervene. It does not tell you whether the workflow is worth doing at all. A perfectly mapped, elegantly automated process that produces something no one needs is still waste — just very legible waste. Strategy decides *which* workflows matter. The anatomy decides how to build them once you've decided they matter. Keep the two jobs separate.

8. Vocabulary

- **Workflow** — a process that turns something into something else through a sequence of steps. The unit of analysis for this entire paper.
- **Input** — what a workflow consumes to begin. Often implicit; naming the hidden ones is half the work.
- **Transformation** — a step that changes input into more valuable output. Deterministic (same input → same output) or judgment-based (context-dependent, fallible).

- **Output** — what a workflow produces. Correctly defined by what the *next step* needs, not by what's convenient to make.
- **Owner** — the single entity accountable for a step. "Single" is required; shared ownership is absent ownership.
- **Handoff** — the moment work and its context pass from one owner to the next. Where workflows most often break. Fails by being dropped, garbled, or delayed.
- **Failure point** — a place a workflow can break, stall, or produce wrong output. Reversible failures are safe to automate; irreversible ones require a human gate.
- **Reversibility** — whether a transformation's failure can be undone by a later step. The dominant factor in what is safe to automate.
- **RAG (retrieval augmented generation)** — retrieving relevant stored facts and supplying them to a generation engine before it produces output. Grounds a transformation in reality.
- **Semantic similarity / vector distance** — closeness in *meaning*, measured as distance between numeric representations. The basis of retrieval. Similarity is not truth.
- **Approval gate** — a required human sign-off before an irreversible step executes. The core governance primitive.
- **Orchestration** — the coordination of a multi-step workflow: sequencing, handoffs, retries, and failure handling, managed as first-class concerns rather than left implicit.

9. Reading guide

Temporal — Workflow concepts · <https://docs.temporal.io/workflows>

Temporal is a production system for running long, multi-step business processes reliably at scale. You are not reading this to learn Temporal the product; you are reading it to see the anatomy of this paper *validated by a system that had no choice but to make it explicit* — because a machine cannot orchestrate a process it cannot represent.

Read it with a translation table in hand:

- Where Temporal describes a **Workflow Definition**, read the input → transformation → output chain you now know. Notice it must be defined precisely, up front. That precision is not overhead; it is the price of reliability.
- Where it describes **Activities** — the individual units of work — read *transformations*. Notice they treat each one as a discrete, named thing with defined inputs and outputs. That is exactly the discipline Section 3 asked of you.
- Where it discusses **retries, timeouts, and failure handling**, read *failure points* — and note how much of a serious orchestration system is dedicated to them. This is the professional's confirmation of our central claim: the failure handling *is* the system, not an afterthought bolted on.
- Where it emphasizes **durability and state** — a workflow that survives crashes and resumes exactly where it left off — read the machine equivalent of our *handoff* problem: how do you pass work and its full context across a boundary without losing anything? Temporal's entire reason to exist is solving that at the boundary between steps.

Read only the conceptual overview — not the SDK tutorials, not the deployment guides. The goal is to recognize your own six concepts wearing an engineer's uniform, not to learn to operate the tool. Twenty minutes, annotating in the margins, is the right dose.

10. Walking questions

Take these on the next walk. They are ordered from "confirm you have it" to "put it to work."

1. Which of the six concepts could you explain, cold, to a skeptical operator right now — and which one would you fumble?
2. Handoffs were named as the place workflows break. Does that match your own experience of things going wrong — or does it contradict where you'd instinctively look first?
3. Pick a workflow you personally run this week. What is its true input — including the tacit one you'd normally forget to name?
4. In that same workflow, which transformations are deterministic and which are judgment-based? Which single step, if it failed, would be *irreversible*?
5. Where does this anatomy already live inside FRIDAY or GBrain — and which slot (input, owner, handoff, failure point) is currently the least well-defined?
6. Name one bounded, high-volume, mostly-reversible workflow where you could test automating a transformation *safely* — small enough to fail without cost, real enough to teach you something.

One-sentence takeaway

Any process is inputs, transformations, and outputs moving between owners across handoffs — and the handoffs, not the steps, are where you should spend your attention, because that is where both value and safety are actually won or lost.